

Sistemes Distribuïts – A Naïve View

Pràctica 1

Nom	Login
Antonio Duran	antonio.d
Arnau González	arnau.gonzalez

Contenido

1 Implementació i arquitectura	3
2 Funcionament de l'algorisme	4
3 Protocol i correcció	5
Per què no es perden increments?.....	6
4 Proves i resultats	6
Interpretació de la sortida	6
5 Rendiment i escalabilitat	7
Cost de comunicació	8
6 Limitacions i incorporació de nodes	8
Limitacions de la solució.....	8
Què canviaria per afegir un node durant l'execució.....	8
7 Conclusions	10

1 Implementació i arquitectura

Hem optat per una arquitectura centralitzada en estrella amb un node coordinador (Node 0) i una cua FIFO d'exclusió mútua. El coordinador custodia el valor real de la variable i decideix qui té el torn en cada moment. Quan un procés vol fer una iteració, demana permís (REQUEST), espera que li toqui (GRANT), llegeix el valor (READ/VALUE) i, si és d'escriptura, envia l'increment (UPDATE). Abans d'alliberar el torn (RELEASE), el mateix coordinador replica la nova dada a tots els nodes i n'espera la confirmació (ACK). A més, cada procés funciona amb un únic fil i gestiona les esperes mitjançant `select()`, permetent complir la pausa d'un segon sense deixar d'atendre els missatges de xarxa en temps real.

L'objectiu principal d'aquesta estructura és assegurar la consistència forta exigida per l'enunciat: tothom ha de tenir el valor correcte abans d'utilitzar-lo. En canalitzar les operacions a través del node 0, s'evita qualsevol condició de cursa, ja que només un node pot accedir a la variable alhora. D'aquesta manera, el protocol garanteix que la dada estigui totalment actualitzada a tota la xarxa abans de cedir el torn al següent participant.

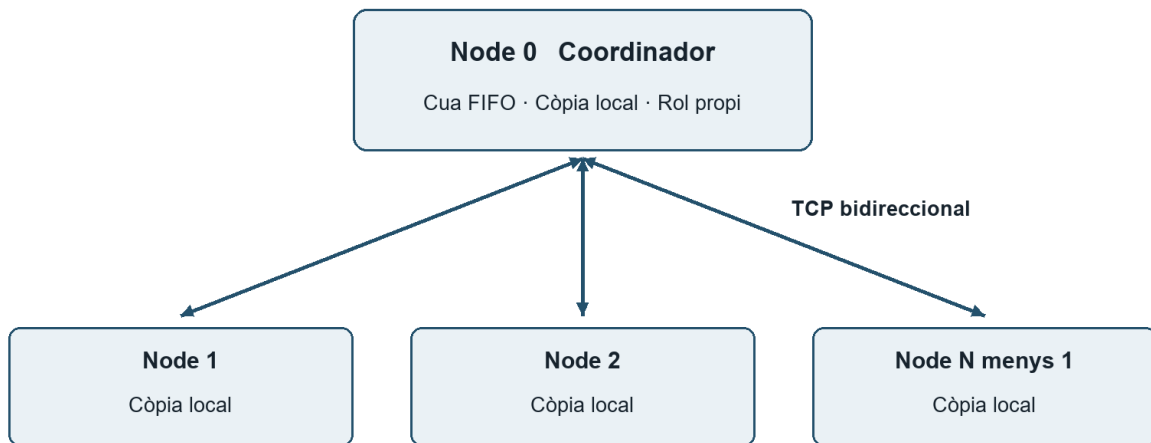


Figura 1. Connexions persistents en estrella. Les fletxes indiquen canals TCP bidireccionals.

Vam descartar que qualsevol node actualitzés pel seu compte i fes difusió a la resta perquè provocaria actualitzacions perdudes (lost updates): si dos processos llegeixen el mateix valor i sumen a la vegada, es trepitgen la feina i les dades queden inconsistents. A més, connectar tots els nodes entre si (malla completa) crearia un excés innecessari de connexions i complicaria molt la coordinació en comparació amb un punt central.

També vam rebutjar un esquema en anell (token ring), ja que el testimoni ha de circular constantment encara que ningú el necessiti, afegint retards innecessaris i fent que la propagació de les noves dades sigui molt més lenta.

Finalment, vam descartar l'ús de múltiples fils (threads) i `sleep()` tradicionals perquè hauria requerit emprar semàfors o mètexs locals (contrari a la restricció de coordinar-se només amb sockets) i congelaria el procés, impedit-li rebre i confirmar les actualitzacions dels altres mentre espera.

Organització del programa i elecció del llenguatge

El sistema s'ha implementat íntegrament en llenguatge C i s'ha estructurat seguint un patró modular dividit en tres capes clarament diferenciades: configuració (`config.c` i `config.h`), que processa els arguments de línia d'ordres, valida la topologia de xarxa i assegura la presència del node coordinador 0; comunicacions (`xarxa.c` i `xarxa.h`), que encapsula la gestió de sockets TCP portables i implementa un protocol binari compacte de 12 bytes gestionat amb la crida al sistema `select()` per atendre missatges de

forma no blocant durant les pauses; i lògica del protocol (main.c), que bifurca el comportament segons el rol per coordinar l'accés exclusiu mitjançant una cua FIFO (Node 0) o sol·licitar torn i replicar el valor com a client (Nodes > 0).

Vam triar C principalment perquè és un llenguatge que ja havíem fet servir en altres assignatures de la carrera per treballar amb sockets, de manera que el teníem molt per la mà i ens resultava més familiar; a més, encaixa perfectament amb el que demana la pràctica, ja que permet fer tota la comunicació directa des de zero sense necessitat d'utilitzar llibreries externes ni eines complexes..

2 Funcionament de l'algorisme

La sincronització comença amb una fase d'arrencada coordinada: en iniciar-se, cada node participant es connecta al coordinador (Node 0) i li envia un missatge READY amb el seu identificador. El coordinador espera que tots els participants configurats estiguin connectats i, un cop ho estan, respon a tothom amb una trama START. Aquesta barrera inicial evita que cap procés comenci a operar o a modificar la dada compartida abans que tots els servidors estiguin a punt per rebre rèpliques.

Un cop arrencat el sistema, l'accés a la variable es regula mitjançant una cua d'exclusió mútua al coordinador. Quan un node vol fer la seva operació, transmet una petició REQUEST. El coordinador encua les sol·licituds per ordre d'arribada (FIFO) i, quan no hi ha cap altre procés a la secció crítica, concedeix el torn enviant un missatge GRANT. Aquest permís cobreix tot el cicle d'operació: el node llegeix el valor vigent mitjançant READ/VALUE i, si és de tipus escriptor, calcula l'increment i envia UPDATE. En canvi, si és un node de només lectura, simplement consulta la dada i no emet cap actualització.

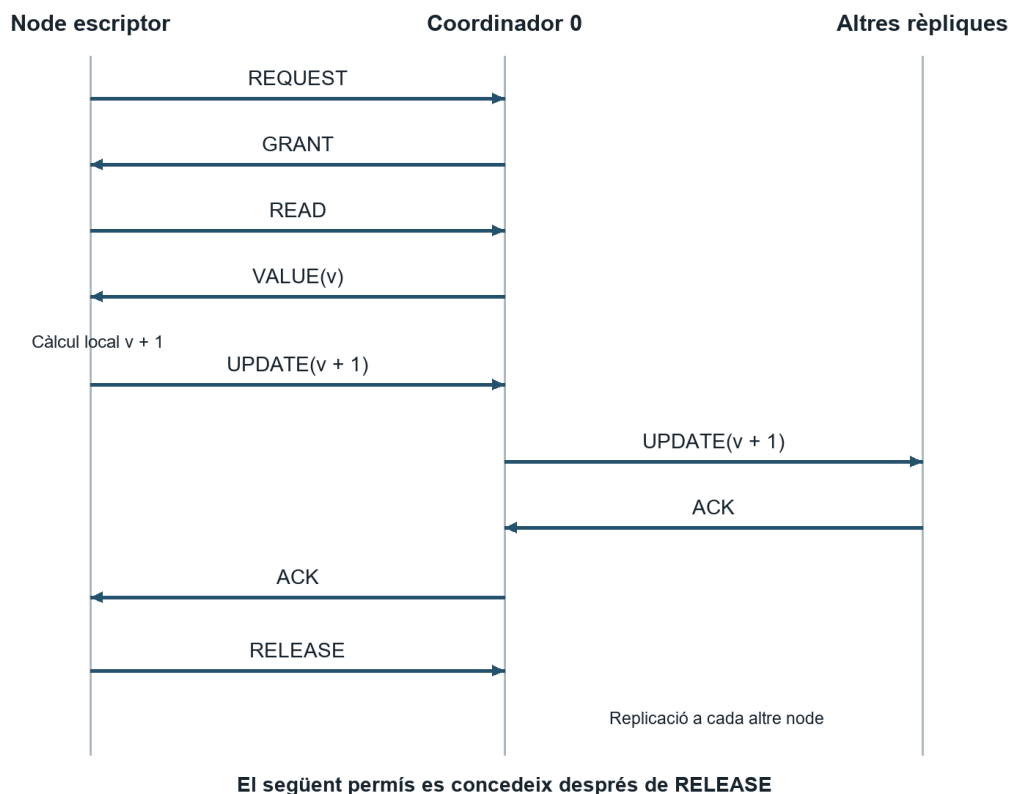


Figura 2. Escripció d'un participant. Un lector omet UPDATE i allibera el permís després de llegir.

Per assegurar que ningú vegi una dada desfasada, el coordinador replica el nou valor a tots els altres nodes abans de donar el torn per tancat. Només quan rep el missatge RELEASE del participant actiu (i després d'haver confirmat totes les rèpliques amb ACK), el coordinador passa a concedir el torn al següent sol·licitant de la cua. A més, el propi coordinador tracta les seves iteracions locals com una petició més que s'afegeix a aquesta mateixa cua, de manera que no hi ha torns fixos ni prioritats artificials entre nodes.

Finalment, la pausa d'un segon entre iteracions s'ha resolt amb un temporitzador controlat mitjançant select(). D'aquesta manera, el procés no es queda congelat i pot continuar rebent i confirmant actualitzacions remotes mentre espera el seu proper torn. Quan un node completa les seves 10 iteracions, avisa el coordinador amb una trama DONE, però es manté escoltant a la xarxa fins que tothom ha acabat. En aquell moment, el coordinador emet un senyal STOP general i es tanquen totes les connexions de forma ordenada.

3 Protocol i correcció

Per estructurar la comunicació sense dependre de formats feixucs ni de llibreries externes, hem dissenyat un protocol binari propi sobre TCP. Cada trama té una mida fixa i compacta de 12 bytes, dividida en tres camps enters de 4 bytes convertits a l'ordre estàndard de xarxa amb htonl() i ntohl(): el tipus de missatge, l'argument associat i la marca de temps de Lamport. Com que TCP transmet un flux continu de bytes (stream) i no assegura el lliurament de paquets complets en una sola lectura, l'intercanvi de dades es gestiona mitjançant un bucle que garanteix la recepció o l'enviament íntegre dels 12 bytes abans de processar el missatge.

La taula següent resumeix el conjunt de trames que formen el protocol del sistema:

Codi	Trama	Argument	Significat
1	READY	ID	Identificar el participant.
2	START	0	Superar la barrera inicial.
3	GRANT	Iteració	Concedir accés exclusiu.
4	READ	0	Demandar el valor actual.
5	VALUE	Valor	Respondre una lectura.
6	UPDATE	Valor absolut	Actualitzar o replicar.
7	ACK	0	Confirmar recepció o aplicació.
8	RELEASE	0	Alliberar l'accés.
9	STOP	0	Finalitzar el procés.
10	REQUEST	Iteració	Demandar accés.
11	DONE	0	Anunciar deu iteracions acabades.

Cal destacar que la trama UPDATE transmet sempre el nou valor absolut calculat localment pel node escriptor, complint la prohibició expressa de la pràctica d'inventar operacions agregades com trames d'increment relatiu (INCREMENT + k) o missatges per declarar el rol del servidor. D'aquesta manera, els nodes lectors es limiten a rebre i aplicar les rèpliques sense originar modificacions pel seu compte.

Per què no es perden increments?

El motiu pel qual mai no es perden increments és la combinació de l'accés exclusiu i la confirmació de les rèpliques. Com que el coordinador només dóna pas a un sol procés a la vegada, el node actiu té la certesa que ningú més tocarà la variable mentre ell treballa. Aquest procés llegeix el valor actual, calcula el següent increment i no allibera el seu torn fins que el coordinador ha enviat la nova dada a tota la xarxa i tothom ha confirmat que l'ha guardat mitjançant un missatge ACK. Com que es comença des de 0 i cada iteració d'escriptura suma exactament 1, si tenim E nodes escriptors fent 10 iteracions cadascun, el resultat final serà forçosament $10 * E$.

D'altra banda, hem incorporat un rellotge lògic de Lamport a cada trama per poder registrar i entendre l'ordre dels esdeveniments a les traces. Cada vegada que un node fa una acció o envia un missatge suma un punt al seu comptador intern, i quan rep una trama de l'exterior actualitza el seu valor agafant el número més alt i sumant-n'hi un. És important subratllar que aquest rellotge no és el que decideix qui té el torn (això ho gestiona directament la cua FIFO del coordinador), sinó que ens serveix com una eina d'observabilitat per comprovar a posteriori que la seqüència de peticions, lectures i actualitzacions s'ha executat amb coherència.

4 Proves i resultats

La verificació s'ha dut a terme sobre Linux mitjançant GCC, utilitzant les opcions de compilació del Makefile (-Wall -Wextra -Werror -pedantic -std=c99) sense cap avís. Les proves s'han executat localment aixecant diversos processos sobre la interfície de xarxa interna (127.0.0.1). A més, s'ha utilitzat Valgrind per comprovar que l'execució allibera correctament tota la memòria dinàmica i no produeix fuites de memòria.

La següent taula resumeix els escenaris que hem provat i el resultat obtingut:

Configuració	Esperat	Observat	Comprovació
1 escriptor	10	10	Deu iteracions i valor final.
3 lectors	0	0	Cap escriptura i còpies iguals.
1 lector i 2 escriptors	20	20	Lectures consistents i rèpliques iguals.
3 escriptors	30	30	Inclou el coordinador escriptor.
6 escriptors	60	60	Cap increment perdut.

Interpretació de la sortida

Pel que fa a la sortida per pantalla, cada procés genera un registre clar de les seves accions de xarxa indicant el temps transcorregut i la marca del rellotge de Lamport:

```

----- RESULTAT -----
Iteracio 10/10 | Llegit: 27 | Escrit: 28 (confirmat)
-----

[Node 1 | 25663 ms | Lamport 873] Rep UPDATE del node 0 | valor: 29
[Node 1 | 25663 ms | Lamport 874] Actualitza el valor local a 29
[Node 1 | 25663 ms | Lamport 875] Envia ACK al node 0 | valor: 0
[Node 1 | 25708 ms | Lamport 902] Rep UPDATE del node 0 | valor: 30
[Node 1 | 25708 ms | Lamport 903] Actualitza el valor local a 30
[Node 1 | 25708 ms | Lamport 904] Envia ACK al node 0 | valor: 0
[Node 1 | 26663 ms | Lamport 905] Envia DONE al node 0 | valor: 0

Iteracions acabades. Esperant que acabin els altres nodes...
[Node 1 | 26709 ms | Lamport 921] Rep STOP del node 0 | valor: 0
[Node 1 | 26709 ms | Lamport 922] Envia ACK al node 0 | valor: 0

=====
RESULTAT FINAL
=====
Node 1: execucio completada.
Iteracions: 10/10
Valor final compartit: 30
=====

```

Figura 3. Sortida per pantalla de la darrera iteració i resultat final compartit.

Aquest format permet seguir fàcilment cada cicle: s'observa com s'assigna el torn, la lectura que fa el node i, en el cas dels escriptors, el valor resultant un cop tots els altres participants han confirmat la rèplica. Al final de l'execució, un resum final mostra que tots els nodes han completat les 10 iteracions i que comparteixen exactament el mateix valor de tancament.

5 Rendiment i escalabilitat

L'anàlisi de trànsit i rendiment mesura el nombre d'esdeveniments SEND registrats i el temps total des de la trama START fins a l'últim ACK de parada. Totes les proves corresponen a entorns on cada procés està configurat en mode READ_WRITE.

La taula següent recull les dades obtingudes en tres proves amb diferent nombre de processos:

Nodes	Valor final	Trames enviades	Durada en segons
1	10	0	10,100
3	30	230	10,115
6	60	875	10,099

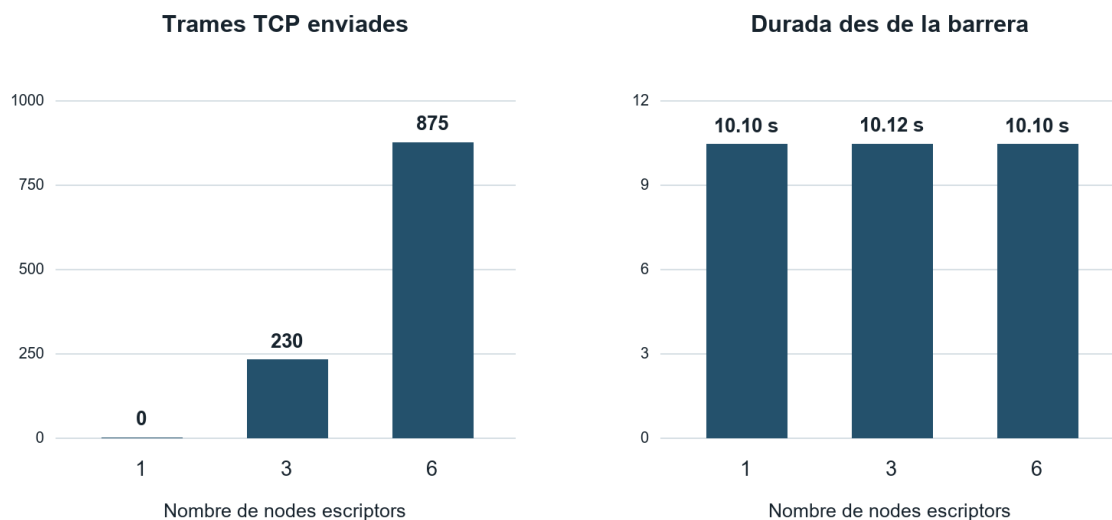


Figura 3. Recompte de trames i durada observada en execucions locals amb tots els nodes escriptors.

Un fet destacable a primer cop d'ull és que la durada total es manté pràcticament constant al voltant dels 10 segons, independentment de si hi ha 1 o 6 nodes. Això és degut al fet que les proves s'han fet en local (amb una latència de xarxa gairebé nul·la) i la pausa d'un segon entre iteracions domina clarament el temps d'execució: mentre un node està complint la seva espera de 1000 ms, els altres aprofiten aquest interval per entrar a la secció crítica i avançar les seves operacions en paral·lel. En una xarxa real o amb un volum molt més gran de servidors, el temps de transferència i les cues començarien a tenir un pes visible.

Cost de comunicació

El nombre de missatges necessaris per a cada operació depèn directament de la mida de la xarxa N :

- Una lectura remota requereix un total de 5 trames (REQUEST, GRANT, READ, VALUE i RELEASE).
- Una escriptura remota necessita aquestes mateixes trames més la parella UPDATE/ACK del propi node, a més de distribuir la rèplica i rebre confirmació de la resta de participants, sumant un total de $5N + 3$ trames. En canvi, si qui escriu és el propi coordinador, el cost baixa a $2(N - 1)$ trames en ser una lectura i càlcul purament locals.

Comptant les deu iteracions de cada participant més les fases inicials de sincronització (READY/START) i de finalització (DONE/STOP/ACK), el cost global creix de forma quadràtica, amb una complexitat de $O(N^2)$. Com que el coordinador centralitza tot el pas de missatges i espera de manera seqüencial els ACK de cada participant per confirmar les rèpliques, aquest esquema es converteix clarament en el coll d'ampolla principal del sistema a mesura que augmenta el nombre de nodes o la latència de la xarxa.

6 Limitacions i incorporació de nodes

Limitacions de la solució

Tot i complir amb els requisits de consistència de l'enunciat, la implementació té una sèrie de limitacions clares derivades del seu enfocament centralitzat i naïve:

- **Punt únic de fallada (Single Point of Failure):** Tot el sistema depèn de la disponibilitat del node coordinador (Node 0). Si aquest procés cau o pateix un error, la xarxa queda completament aturada, ja que cap altre participant pot sol·licitar torns ni coordinar noves rèpliques.
- **Absència de recuperació i consens davant de caigudes:** El protocol assumeix que cap node fallarà durant l'execució. Si un participant es desconnecta sobtadament mentre el coordinador està en plena fase de replicació (UPDATE), aquest es quedarà bloquejat o reportarà un error de connexió, deixant les dades potencialment desincronitzades.
- **Serialització ineficient de les lectures:** La cua FIFO no distingeix entre lectors i escriptors a l'hora de concedir l'accés exclusiu. Això obliga a atendre els nodes READ_ONLY d'un en un en comptes de permetre lectures concurrents, creant un coll d'ampolla innecessari quan augmenta el nombre de processos de només lectura.
- **Coll d'ampolla i cost quadràtic:** Totes les trames han de passar obligatòriament pel coordinador. Com que en cada escriptura cal avisar a tots els nodes d'un en un i esperar el seu ACK de forma seqüencial, el cost de missatges creix de forma quadràtica ($O(N^2)$), penalitzant fortament la latència quan el nombre de processos creix.

Què canviaria per afegir un node durant l'execució

En la nostra implementació actual, la composició del sistema és completament estàtica: el coordinador rep la llista d'adreces a l'inici, accepta tots els participants i tanca immediatament el socket d'escolta (central->escolta) abans d'emetre el senyal START. A més, les estructures de control (com ara la cua de

peticions o la taula de connexions) tenen una capacitat fixa reservada exclusivament per als nodes inicials.

Per permetre la incorporació en calent d'un nou procés a mig camí, no n'hi hauria prou amb donar-li la IP del coordinador, sinó que caldria modificar l'arquitectura seguint aquests passos:

1. **Mantenir obert el canal d'admissió:** El coordinador hauria de conservar el socket d'escolta actiu dins del conjunt del select() durant tota l'execució, i caldria definir una nova trama de protocol (per exemple, JOIN) perquè el nou sol·liciti unir-se a la xarxa.
2. **Pausa temporal de la cua:** En rebre una petició d'admissió, el coordinador hauria d'esperar que l'operació que estigui en curs acabi i congelar temporalment la concessió de nous torns (GRANT) per evitar inconsistències mentre es reconfigura el sistema.
3. **Redimensionament de memòria:** Caldria ampliar dinàmicament mitjançant realloc els vectors interns del coordinador (connexions, estat, iteracions i cua) per poder allotjar el nou descriptor de socket i seguir el seu progrés.
4. **Sincronització de l'estat inicial:** Abans de deixar-lo operar, el coordinador hauria d'enviar el valor vigent de la variable al nou node i esperar un missatge de confirmació (ACK), assegurant així la restricció de l'enunciat que cap participant pot operar amb una còpia antiga o incorrecta.
5. **Integració a les rèpliques i represa:** Un cop confirmat, el node nou s'afegiria com a destinatari obligatori dels futurs missatges UPDATE de replicació i se li permetria començar a demanar torn amb REQUEST, reprenent el funcionament normal de la cua FIFO.
6. **Adaptació del criteri de finalització:** Caldria decidir si el nou participant ha de fer les 10 iteracions completes o només les que quedin, i actualitzar el comptador global de nodes finalitzats (acabats) perquè el coordinador no emeti l'ordre d'aturada (STOP) abans que aquest node hagi enviat el seu DONE.

7 Conclusions

Aquesta pràctica ens ha servit per veure a la realitat conceptes que havíem explicat a classe, com ara l'exclusió mútua, com mantenir les dades al dia entre diferents màquines i com ordenar els missatges. Com que no podíem fer servir eines del mateix sistema operatiu com la memòria compartida o els semàfors, hem hagut de fer-ho tot amb sockets TCP des de zero. Gràcies a això ens hem adonat que una cosa tan senzilla com llegir i sumar un a una variable es complica molt quan hi ha diversos processos treballant alhora, ja que si no es coordinen bé es trepitgen la feina i es perden dades.

L'opció de posar un coordinador central amb una cua per torns ens ha demostrat que és una manera molt clara d'evitar aquests problemes: com que només un node pot tocar la variable a la vegada i no es passa al següent fins que tothom ha rebut el canvi, la dada no es desquadra mai. Però també hem comprovat el desavantatge d'això: el coordinador acaba fent tota la feina, s'envien moltíssims missatges quan s'afegeixen més nodes i es crea un coll d'ampolla evident. D'altra banda, utilitzar el rellotge de Lamport ens ha ajudat a entendre de forma pràctica com podem ordenar què passa abans i què passa després a través dels missatges, sense dependre de l'hora de l'ordinador.

Finalment, fer aquest exercici ens ha fet veure que dissenyar un sistema distribuït de veritat va molt més enllà de quan tot va bé. El nostre codi funciona amb la configuració prevista, però si un ordinador es desconnecta per sorpresa o si volguéssim afegir-ne un de nou mentre el programa ja està en marxa, el sistema no ho sabria gestionar i es quedaria penjat, cosa que demostra la importància que tenen els mecanismes de recuperació i tolerància a fallades.